

Appointment Booking Application

This document defines the functional and non-functional requirements for the Appointment Booking Application, serving as the authoritative reference for all stakeholders throughout the development lifecycle.

Version 1.0

April 2026

Table of Contents

1. Introduction	3
1.1 Purpose	3
1.2 Context	3
1.3 Objectives	3
1.4 Scope	3
1.5 Definitions and Acronyms	4
2. Description of Actors	4
3. Functional Requirements	5
3.1 User Registration (FR-01)	5
3.2 User Authentication (FR-02)	5
3.3 Slot Creation by Admin (FR-03)	5
3.4 Browse Available Slots (FR-04)	5
3.5 Book an Appointment (FR-05)	5
3.6 Cancel an Appointment (FR-06)	6
3.7 View Booking History (FR-07)	6
3.8 Admin Schedule Overview (FR-08)	6
3.9 Slot Deletion by Admin (FR-09)	6
3.10 Email Notifications (FR-10)	6
4. Non-Functional Requirements	7

5. Technical Constraints	8
6. Design Diagrams	8
6.1 Use Case Diagram	8
6.2 High-Level Data Flow — Booking Process	9
7. Provisional Planning	10
8. Risks and Mitigation Strategies	10

1. Introduction

1.1 Purpose

This document defines the functional and non-functional requirements for the Appointment Booking Application. It serves as the authoritative reference for developers, designers, testers, and project managers throughout the development lifecycle. The specification outlines the expected system behavior, constraints, and quality attributes that must be satisfied for the application to be considered complete and ready for deployment. All stakeholders should refer to this document as the single source of truth for requirements scope and acceptance criteria.

1.2 Context

Manual appointment scheduling through phone calls, spreadsheets, or paper logs is error-prone, time-consuming, and unavailable outside business hours. Common problems include double-bookings, no-shows, inconsistent availability information, and excessive administrative overhead. These issues lead to wasted resources, frustrated clients, and revenue loss for service providers who rely on efficient scheduling to maximize their utilization rates.

This application digitizes the entire appointment lifecycle — from defining available time slots through booking, confirmation, modification, and cancellation — providing a self-service experience for clients and a centralized management interface for administrators. By replacing manual processes with an automated, rule-based system, the application eliminates the most common sources of scheduling error while improving accessibility and transparency for all users.

1.3 Objectives

- Eliminate double-bookings through real-time slot locking and database-level transaction isolation.
- Provide 24/7 availability so clients can book at any time without human intervention.
- Reduce administrative burden by automating schedule creation, conflict detection, and booking confirmations.
- Ensure data integrity with secure authentication, encrypted storage, and audit-ready logging.
- Deliver a responsive experience that works on desktops, tablets, and mobile devices.

1.4 Scope

In Scope: User registration and authentication; admin-defined time slot management; real-time availability browsing; appointment booking with confirmation; booking cancellation; admin dashboard for schedule oversight; email notification on booking events.

Out of Scope: Online payment processing; video conferencing integration; multi-provider marketplace; SMS notifications; calendar sync (Google/Outlook); analytics and reporting dashboards; native mobile

applications.

1.5 Definitions and Acronyms

Term	Definition
SRS	Software Requirements Specification
JWT	JSON Web Token — used for stateless authentication
REST	Representational State Transfer — API architectural style
RBAC	Role-Based Access Control
HTTPS	Hypertext Transfer Protocol Secure

Table 1: Definitions and Acronyms

2. Description of Actors

The system involves three distinct actors, each with specific roles and access privileges. Understanding these actors is essential for defining functional requirements and access control policies. The two primary actors — Client and Administrator — interact directly with the system through its user interface, while the System actor operates autonomously to enforce business rules and deliver notifications.

Actor	Type	Responsibilities	Key Interactions	Access Level
Client	Primary	Registers, browses available slots, books and cancels appointments, manages personal profile.	Login, View Slots, Book, Cancel, View History	Standard (own data only)
Administrator	Primary	Defines service availability, creates time slots, views all bookings, manages the schedule, handles cancellations.	Login, Create Slots, View All Bookings, Delete Slots, Manage Schedule	Elevated (full system access)
System	Secondary	Sends email confirmations, enforces booking rules, prevents conflicts, manages session tokens.	Auto-notify, Lock Slots, Expire Sessions	Internal (automated)

Table 2: Actor Descriptions

3. Functional Requirements

3.1 User Registration (FR-01)

The system shall allow new users to create an account using a valid email address and a password meeting minimum complexity requirements (8+ characters, at least one uppercase letter and one number). A confirmation email must be sent before the account is activated. The registration process should validate email uniqueness in real-time and provide clear feedback on password strength. Upon successful registration, the user receives a verification email containing an activation link that expires after 24 hours. Unverified accounts are automatically purged after 7 days to maintain database hygiene.

3.2 User Authentication (FR-02)

The system shall authenticate users via email and password. Upon successful login, a signed JWT shall be issued with a configurable expiry period (default: 1 hour for access tokens, 7 days for refresh tokens). Failed login attempts shall be rate-limited to a maximum of 5 attempts per 15-minute window per IP address. After exceeding this threshold, the account is temporarily locked and the user receives a notification email. The authentication middleware validates the JWT signature and expiry on every protected API request, ensuring that expired or tampered tokens are rejected immediately.

3.3 Slot Creation by Admin (FR-03)

Administrators shall be able to define available time slots by specifying a date, start time, end time, and slot capacity. The system must validate slots to prevent overlaps with existing slots for the same date. When a new slot conflicts with an existing one, the API returns a 409 Conflict response with details of the overlapping slot. Administrators may also create recurring slots (e.g., every Monday from 9:00 to 12:00) to reduce repetitive data entry. Each slot records its creator, creation timestamp, and current booking count for audit purposes.

3.4 Browse Available Slots (FR-04)

Clients shall be able to view all available (unfilled) time slots sorted by date and time. The interface must indicate remaining capacity for each slot and display fully booked slots as unavailable. The slot listing supports date-range filtering and pagination to handle large datasets efficiently. Slot availability data is served from a read-optimized query with indexed columns on date and status, ensuring sub-200ms response times even with thousands of slot records. The display distinguishes between partially booked slots (showing remaining capacity) and fully booked slots (grayed out or hidden based on user preference).

3.5 Book an Appointment (FR-05)

A logged-in client shall be able to book an available slot. The system must verify slot availability at the moment of booking using a database-level lock (SELECT ... FOR UPDATE within a transaction) to prevent race conditions when multiple clients attempt to book the same slot simultaneously. A confirmation must be displayed to the client and an email sent to the registered address. The booking record includes the client ID, slot ID, booking timestamp, and an initial status of "confirmed." If the slot becomes fully booked during the transaction, the system returns a clear error message and the client is prompted to select another slot.

3.6 Cancel an Appointment (FR-06)

Clients may cancel their own bookings at any time. Upon cancellation, the slot capacity must be incremented immediately, making it available for other clients. A cancellation confirmation email shall be sent to the client. The booking status transitions from "confirmed" to "cancelled" and the cancellation timestamp is recorded. Administrators may also cancel bookings on behalf of clients, in which case both the client and the admin receive confirmation emails. Cancelled bookings remain in the system for historical reference and are not physically deleted.

3.7 View Booking History (FR-07)

Clients shall access a list of all their past and upcoming bookings, including status (confirmed, cancelled), date, time, and slot details. The booking history view supports filtering by status and date range, and presents records in reverse chronological order (most recent first). Administrators shall see all bookings across all clients, with the ability to filter by client name, date range, and status. The history view implements server-side pagination to maintain performance as the dataset grows over time.

3.8 Admin Schedule Overview (FR-08)

The admin dashboard shall display a consolidated view of all slots and their booking status (empty, partial, full) for any selected date range. The overview provides at-a-glance visibility into schedule utilization, helping administrators identify underutilized time periods and plan capacity adjustments. The dashboard includes summary statistics such as total slots, total bookings, and average fill rate for the selected period. The schedule view updates in near-real-time, reflecting new bookings within 30 seconds of creation.

3.9 Slot Deletion by Admin (FR-09)

Administrators may delete a slot only if no bookings are associated with it. If bookings exist, the system must reject deletion and prompt the admin to cancel individual bookings first. This safeguard prevents accidental data loss and ensures that clients with confirmed bookings are not left without notification. The deletion is a soft delete — the slot is marked as inactive rather than removed from the database, preserving referential integrity for historical reporting and audit trails.

3.10 Email Notifications (FR-10)

The system shall automatically send email notifications for: successful registration, booking confirmation, booking cancellation, and optionally upcoming appointment reminders as configured by the admin. Email delivery is handled asynchronously via a job queue to prevent API response delays. Failed email deliveries are retried up to 3 times with exponential backoff. All email events (sent, failed, bounced) are logged for troubleshooting. In-app notifications serve as a fallback when email delivery fails, ensuring the client always receives timely information about their booking status.

4. Non-Functional Requirements

Non-functional requirements define the quality attributes and operational constraints of the system. These requirements are as critical as functional requirements because they determine the system's reliability, performance, and user experience under real-world conditions. Each requirement includes a measurable target to enable objective validation during testing and acceptance.

Category	Requirement	Target
Security	Passwords hashed with bcrypt (cost factor 12+). JWT tokens with RS256 signing. Input sanitization against XSS and SQL injection. HTTPS enforced on all endpoints.	No plaintext credentials in storage or transit.
Performance	API response time under 500ms at the 95th percentile. Slot listing queries optimized with indexed columns. Frontend initial load under 2 seconds on 3G.	p95 latency < 500ms.
Usability	Responsive layout supporting 320px to 2560px viewports. WCAG 2.1 Level AA compliance. Booking flow completable in 3 or fewer clicks from slot selection.	3-click booking path.
Availability	Target uptime of 99.5% excluding scheduled maintenance. Graceful degradation with user-friendly error pages. Automated health checks.	99.5% uptime.
Scalability	Database schema designed to handle 10,000+ bookings without performance degradation. Stateless API layer for horizontal scaling. Connection pooling enabled.	10K+ booking records.
Compatibility	Support for the latest two major versions of Chrome, Firefox, Safari, and Edge. Progressive enhancement for older browsers.	Last 2 major versions.

Table 3: Non-Functional Requirements

5. Technical Constraints

The following technology stack and architectural constraints govern the implementation. These choices reflect a balance between developer productivity, ecosystem maturity, and operational simplicity. All team members must adhere to these constraints unless a formal change request is approved by the project lead.

Layer	Technology	Version	Notes
Frontend	React.js	v18+	Functional components with hooks. No server-side rendering.
Backend	Node.js with Express	Node v20 LTS, Express v4.18+	REST API with JSON request/response.
Database	MySQL	v8.0+	Relational schema with indexed queries and transaction support.

Table 4: Technology Stack

Additional Constraints:

- Deployment on a Linux-based hosting environment (e.g., AWS EC2, DigitalOcean Droplet).
- Environment variables for all configuration — database credentials, JWT secrets, and email service keys must never be hardcoded.
- Version control via Git with a main/develop/feature branching strategy.
- API documentation generated using the OpenAPI 3.0 specification format.

6. Design Diagrams

6.1 Use Case Diagram

The use case diagram below illustrates the interactions between the primary actors (Client and Administrator) and the system's core functions. The Client actor can perform self-service operations such as browsing slots, booking appointments, and managing their booking history. The Administrator actor has elevated privileges for slot management and schedule oversight. Both actors share the authentication use case (Login / Register), which is included by all other use cases as a prerequisite. Dashed lines represent «include» and «extend» relationships between use cases.

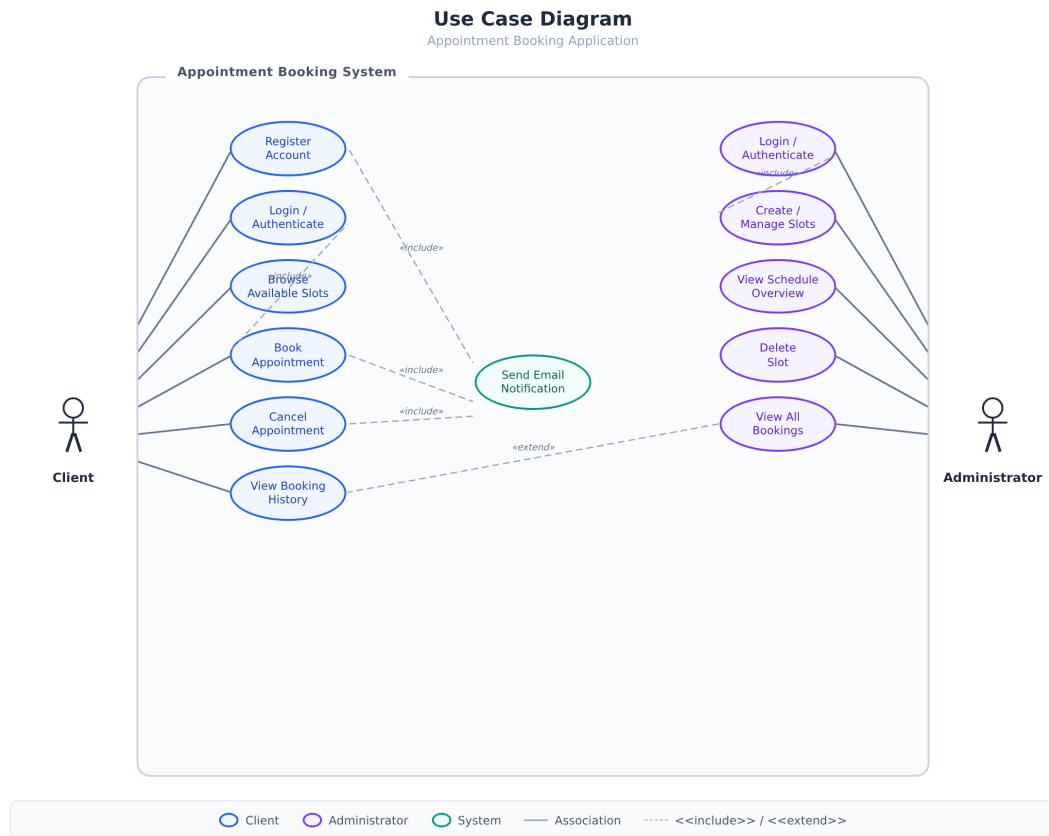


Figure 1: Use Case Diagram — Appointment Booking Application

6.2 High-Level Data Flow — Booking Process

The booking process data flow involves four components: the Client Browser, the API Server (Express.js), the MySQL Database, and the Email Service (SMTP via Nodemailer). The client sends an HTTP request to the API server, which executes a SQL query against the database. The result set is returned as a JSON response to the client. Concurrently, certain operations (booking confirmation, cancellation) trigger the email service to send notifications. All database operations are wrapped in transactions to ensure atomicity, and the API server enforces authentication and authorization before processing any protected endpoint.

Step	From	To	Data
1	Client (Browser)	API Server	HTTP Request (POST /api/bookings)
2	API Server	MySQL Database	SQL Query (INSERT with row lock)
3	MySQL Database	API Server	Result Set (booking confirmation)
4	API Server	Client (Browser)	JSON Response (201 Created)

5	API Server	Email Service	Trigger (send confirmation email)
---	------------	---------------	-----------------------------------

Table 6: Booking Process Data Flow

7. Provisional Planning

The project follows a phased development approach spanning seven weeks. Each phase has defined deliverables and completion criteria. The cumulative timeline ensures that dependencies between phases are respected, and that each phase builds upon the validated output of the previous one. Buffer time is implicitly included in the integration and testing phase to accommodate any spillover from earlier phases.

Phase	Duration	Cumulative	Key Tasks
Phase 1: Analysis and Requirements	1 Week	Week 1	Stakeholder interviews, requirement elicitation, SRS finalization, project setup (Git repo, dev environment), technology validation.
Phase 2: Design and UI/UX	1 Week	Week 2	Wireframing, database schema design (ER diagram), API endpoint specification (OpenAPI), component architecture planning.
Phase 3: Backend Development	2 Weeks	Week 3-4	Express server setup, MySQL schema implementation, REST API endpoints (auth, slots, bookings), JWT middleware, input validation, unit tests.
Phase 4: Frontend Development	2 Weeks	Week 5-6	React scaffolding, authentication pages, slot browsing UI, booking flow components, admin dashboard, responsive styling, API integration.
Phase 5: Integration, Testing, Deployment	1 Week	Week 7	End-to-end testing, cross-browser validation, performance profiling, bug fixes, production build, server deployment, handover documentation.

Table 7: Provisional Project Plan

8. Risks and Mitigation Strategies

Risk management is integral to the project's success. The following table identifies the most significant risks, their severity, and the proposed mitigation strategies. Each risk has been evaluated based on its likelihood and potential impact, and the mitigation strategies are designed to either prevent the risk from materializing or minimize its consequences if it does.

#	Risk	Severity	Proposed Solution
1	Double booking under concurrent load	High	Use database-level row locking (SELECT ... FOR UPDATE) within a transaction. Implement optimistic concurrency control with version columns as a secondary safeguard.
2	Data breach or unauthorized access	High	Enforce HTTPS/TLS on all endpoints. Hash passwords with bcrypt. Use short-lived JWTs with refresh token rotation. Apply RBAC middleware on all protected routes.
3	Performance degradation at scale	Medium	Create composite indexes on frequently queried columns (date, status). Implement connection pooling. Add caching for slot availability queries. Set up query performance monitoring.
4	Scope creep during development	Medium	Strictly adhere to the approved SRS. Any new requirement must go through a formal change request process with impact analysis before acceptance.
5	Email delivery failures	Low	Use a reliable transactional email provider (e.g., SendGrid, Mailgun). Implement retry logic with exponential backoff. Log all email events. Show in-app confirmations as fallback.
6	Cross-browser compatibility issues	Low	Use standard CSS features with progressive enhancement. Run automated cross-browser tests. Avoid bleeding-edge APIs without polyfills.

Table 8: Risks and Mitigation Strategies